

# Efficient Pipeline Parallelism For Reinforcement Learning

Faiz Sameer Ahmed

## Advisor

Genya Ishigaki, genya.ishigaki@sjsu.edu

## Committee Members

Fabio Di Troia, fabio.ditroia@sjsu.edu

Jelena Gligorijevic, jelena.gligorijevic@sjsu.edu

## Introduction

Reinforcement Learning (RL) has proven effective for complex tasks like game playing and robotics. As models grow larger, training them requires distributing computation across multiple devices. Pipeline parallelism addresses this by splitting the neural network into stages, with each device handling different layers.

The challenge with pipeline parallelism is communication cost—activations and gradients must be sent between devices during both forward and backward passes. This project explores gradient compression techniques to reduce this data transfer while preserving training performance.

**Keywords:** *Reinforcement Learning, Pipeline Parallelism, Gradient Compression, Distributed Training*

## Problem Definition and Motivation

When training RL models using pipeline parallelism, communication between devices becomes expensive. Activations flow forward and gradients flow backward through the pipeline, and in RL training with actor-critic networks, these exchanges happen frequently. The gradient tensors can be large, consuming bandwidth and slowing down training.

While gradient compression works well for data-parallel training, it hasn't been thoroughly explored for pipeline-parallel RL. This matters because as RL models grow, we need efficient ways to train them across multiple devices. Reducing communication overhead means faster training, lower costs, and making distributed RL practical in bandwidth-constrained settings like edge computing or resource-limited clusters.

## Results Achieved in CS 297

- Set up distributed training environment and measured baseline performance on Car Racing Gym
- Built pipeline-parallel training system to track network usage and training metrics

- Tested statistical gradient compression—it reduced communication but slowed convergence enough to cancel the gains
- Implemented gradient accumulation method that showed better results
- Created monitoring infrastructure with WandB, Docker containers, and data collection tools

## Expected Deliverables in CS 298/299

- Implementation of threshold-based gradient compression with adaptive scheduler
- Performance comparison report analyzing threshold-based vs. p90 percentile approaches
- Model convergence quality validation framework using frozen model checkpoints
- Extended implementation supporting two additional RL environments (Atari Pong and LunarLander)
- Final thesis report documenting methodology, experiments, and findings

## Timeline and Milestones for CS 298/299

|                             | January | February | March | April | May |  |
|-----------------------------|---------|----------|-------|-------|-----|--|
| Threshold-based Approach    |         |          |       |       |     |  |
| Evaluations & Model Quality |         |          |       |       |     |  |
| Additional Environments     |         |          |       |       |     |  |
| Report Writing              |         |          |       |       |     |  |
| Defense Preparation         |         |          |       |       |     |  |

## References

- [1] A. Nair, P. Srinivasan, S. Blackwell, C. Alcicek, R. Fearon, A. De Maria, V. Panneershelvam, M. Su-leyman, C. Beattie, S. Petersen, *et al.*, “Massively parallel methods for deep reinforcement learning,” in *ICML Deep Learning Workshop*, 2015.
- [2] L. Espeholt, H. Soyer, R. Munos, K. Simonyan, V. Mnih, T. Ward, Y. Doron, V. Firoiu, T. Harley, I. Dunning, *et al.*, “Impala: Scalable distributed deep-rl with importance weighted actor-learner architectures,” in *Proc. ICML*, pp. 1407–1416, 2018.
- [3] D. Horgan, J. Quan, D. Budden, G. Barth-Maron, M. Hessel, H. Van Hasselt, and D. Silver, “Distributed prioritized experience replay,” in *Proc. ICLR*, 2018.
- [4] W. Chen, S. Wang, Y. Hong, and G. B. Giannakis, “Communication-efficient policy gradient methods for distributed reinforcement learning,” *IEEE Trans. Signal Process.*, vol. 69, pp. 2629–2641, 2021.
- [5] Y. Lin, S. Han, H. Mao, Y. Wang, and W. J. Dally, “Deep gradient compression: Reducing the communication bandwidth for distributed training,” in *Proc. ICLR*, 2018.
- [6] D. Alistarh, D. Grubic, J. Li, R. Tomioka, and M. Vojnovic, “Qsgd: Communication-efficient sgd via gradient quantization and encoding,” in *Proc. NeurIPS*, pp. 1709–1720, 2017.

- [7] Y. Huang, Y. Cheng, A. Bapna, O. Firat, D. Chen, M. Chen, H. Lee, J. Ngiam, Q. V. Le, Y. Wu, and Z. Chen, “Gpipe: Efficient training of giant neural networks using pipeline parallelism,” in *Proc. NeurIPS*, pp. 103–112, 2019.
- [8] D. Narayanan, A. Harlap, A. Phanishayee, V. Seshadri, N. R. Devanur, G. R. Ganger, P. B. Gibbons, and M. Zaharia, “Pipedream: Generalized pipeline parallelism for dnn training,” in *Proc. ACM SOSP*, pp. 1–15, 2019.
- [9] M. Shoeybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, and B. Catanzaro, “Megatron-lm: Training multi-billion parameter language models using model parallelism,” *arXiv preprint arXiv:1909.08053*, 2019.
- [10] S. Rajbhandari, J. Rasley, O. Ruwase, and Y. He, “Zero: Memory optimizations toward training trillion parameter models,” in *Proc. IEEE SC*, pp. 1–16, 2020.
- [11] H. Lim, Y. J. Kim, and M. L. Sichitiu, “Federated reinforcement learning for training control policies on multiple iot devices,” *Sensors*, vol. 20, no. 5, p. 1359, 2020.
- [12] X. Feng, C. Luo, J. Chen, Y. Huang, J. Zhang, W. Xu, J. Li, and V. C. M. Leung, “Iotsl: Toward efficient distributed learning for resource-constrained internet of things,” *IEEE Internet Things J.*, vol. 10, no. 11, pp. 9892–9905, 2023.